



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

PHƯƠNG PHÁP PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG VỚI UML

BỘ MÔN CÔNG NGHỆ PHẦN MỀM



Giới thiệu nội dung học phần

Bài 1: Giới thiệu quy trình ICONIX Process

Bài 2: Mô hình hóa miền (Domain modeling)

Bài 3: Mô hình hóa Use Case (Use Case Modeling)

Bài 4: Requirements Review – Kiểm duyệt yêu cầu → **Kiểm tra TX1**

Bài 5: Robustness Analysis — Cầu nối từ Use Case đến Thiết kế

Bài 6: Preliminary Design Review — Kiểm duyệt thiết kế sơ bộ → **Kiểm tra TX 2**

Bài 7: Technical Architecture — Kiến trúc kỹ thuật

Bài 8: Sequence Diagrams

Bài 9: Critical Design Review (CDR) → Kiểm tra TX3



TẦM QUAN TRỌNG CỦA MÔN HỌC

Môn học này có quan trọng không → Có. Vì:

- Nó là tài liệu bắt buộc trong đề tài, đồ án, dự án...
- Nó là cơ sở để cho các lập trình viên viết code
- Nó là tài liệu để cho các mô hình AI hiện đại sinh mã nguồn, chương trình...



BÀI 9: CRITICAL DESIGN REVIEW (CDR)

Về kiến thức

-  Giới thiệu Critical Design Review (CDR)
-  Vai trò trong ICONIX
-  Quy trình 5 bước
-  10 nguyên tắc thực hiện
-  Lỗi thường gặp
-  Từ thiết kế → mã nguồn (EF Core)

Về kỹ năng

-  Hiểu vai trò & thời điểm CDR
-  Áp dụng 10 nguyên tắc kiểm tra
-  Nhận diện lỗi thiết kế phổ biến
-  Đối chiếu Use Case – Sequence – Class
-  Chuyển thiết kế thành code EF Core



Bối cảnh và vai trò của CDR



Mốc kỹ thuật bắt buộc



Sau Sequence, trước Coding



Kiểm tra tam giác Use–Seq–Class



Phát hiện lỗi sớm



“Lưới lọc cuối cùng”

Vai trò của các thành viên



Kiến trúc sư: kiểm tra layering



Developer: đánh giá khả năng code



Tester: soi nhánh lỗi



BA: đối chiếu yêu cầu



Moderator: điều phối buổi review



Quy trình CDR (Tổng quan)

- 1 Chuẩn bị tài liệu
- 2 Kiểm tra Use Case
- 3 Kiểm tra Sequence Diagram
- 4 Kiểm tra Class Diagram
- 5 Tổng hợp & ra quyết định



Bước 1: Chuẩn bị tài liệu



Use Case,
Sequence,
Class đầy đủ



Kiến trúc
cập nhật



In hoặc
trình chiếu
song song



Phân công
vai trò



Không có
khách hàng
tham dự

Bước 2: Rà soát Use Case



Đọc to
Use Case



Xác nhận
basic &
alternate flow



Phát hiện
thiếu logic



Kiểm tra
điều kiện
biên



Làm
chuẩn để soi
diagram

Bước 3: Kiểm tra Sequence Diagram



So khớp với Use Case



Kiểm tra alt/opt/loop



Vi phạm layering?



Tên message rõ chưa?



Logic có liền mạch?

Bước 4: Kiểm tra Class Diagram



Mỗi message $\leftrightarrow$ 1 method



Thiếu = lỗi thiết kế



Method không gọi = code chết



Kiểm tra quan hệ lớp



Bổ sung hoặc gỡ bỏ hợp lý



Bước 5: Tổng hợp và quyết định



Lập danh sách lỗi



Phân loại nghiêm trọng / nhẹ



Giao nhiệm vụ chỉnh sửa



Chỉ code khi lỗi đã xử lý



“Qua cửa” mới lập trình



Tam giác vàng ICONIX

▲ Use
Case

▲
Sequence
Diagram

▲ Class
Diagram

⚙ Mọi góc
phải khớp

🧱 Thiếu =
lỗi thiết kế

10 Nguyên tắc CDR (Tổng quan)



Tập trung vào logic



Bám sát Use Case



Kiểm tra tam giác vàng



Đa góc nhìn



Không code trước review

Nguyên tắc 1: Không bỏ qua Use Case



Mọi kiểm tra dựa vào Use Case



Không xét riêng biểu đồ



Review theo logic người dùng



Câu hỏi: “Hệ thống làm gì?”



Giữ traceability toàn vẹn



Nguyên tắc 2: Kiểm tra Alternate Flow



Không chỉ “happy path”



Bao phủ lỗi, ngoại lệ



Mỗi alternate có controller riêng



Sequence phụ cho alt-flow



Kiểm tra qua test case



Nguyên tắc 3: Giữ đúng phân tầng



Boundary → Controller → Entity



Không bỏ tầng



Không gọi ngược



Một chiều duy nhất



Kiểm tra dependency kỹ

Nguyên tắc 4: Tên method rõ nghĩa



Dùng động từ + danh từ



Thống nhất với Sequence



Không “doStuff()”, “handle()”



Phản ánh hành vi nghiệp vụ



Đọc tên là hiểu việc

Nguyên tắc 5: Đảm bảo logic đầy đủ



Không bỏ bước xử lý



Mỗi thao tác đều có phản hồi



Input → output rõ ràng



Message return thể hiện rõ



Đảm bảo “data flow khép kín”

**Nguyên tắc 6:
Không thêm
chi tiết kỹ
thuật**



Không bàn thuật toán



Không tối ưu sớm



Tập trung hành vi logic



CDR $\neq$ Code Review



Giữ ở mức mô hình



Nguyên tắc 7: Cập nhật tài liệu ngay

 Mọi
chỉnh sửa
phải ghi lại

 Class,
Sequence
đồng bộ

 Update
PDR notes

 Lưu
version mới

 Tránh
drift mô
hình—code

Nguyên tắc 8: Testability và Security



Dễ test
từng tầng



Kiểm
soát truy cập



Không lộ
dữ liệu



Mock
dependency
bằng DI



Review
cả phi chức
năng



Nguyên tắc 9: Giữ teamwork



Review = hợp tác, không phán xét



Tôn trọng vai trò



Kiến trúc sư định hướng



Tester phản biện



Ghi lại quyết định chung

Nguyên tắc 10: “Qua cửa” mới code



CDR = cổng sang coding



Fix xong lỗi mới triển khai



Tài liệu phải đồng nhất



Không bỏ qua checklist



Kỷ luật kỹ thuật ICONIX

Các lỗi thường gặp



Không
đối chiếu
diagram



Vi phạm
kiến trúc



Tên
method mơ
hồ



Bỏ qua
alt flow



Diagram
không cập
nhật

Chuyển thiết kế sang mã nguồn (EF Core)



Dùng Entity Framework Core



Mỗi Class $\leftrightarrow$ Table



Mỗi Operation $\leftrightarrow$ Method



Sinh scaffold code



Từ mô hình $\rightarrow$ code thật

Ví dụ EF Core Scaffold



ReviewService → Class thật



DbContext, DbSet<T>



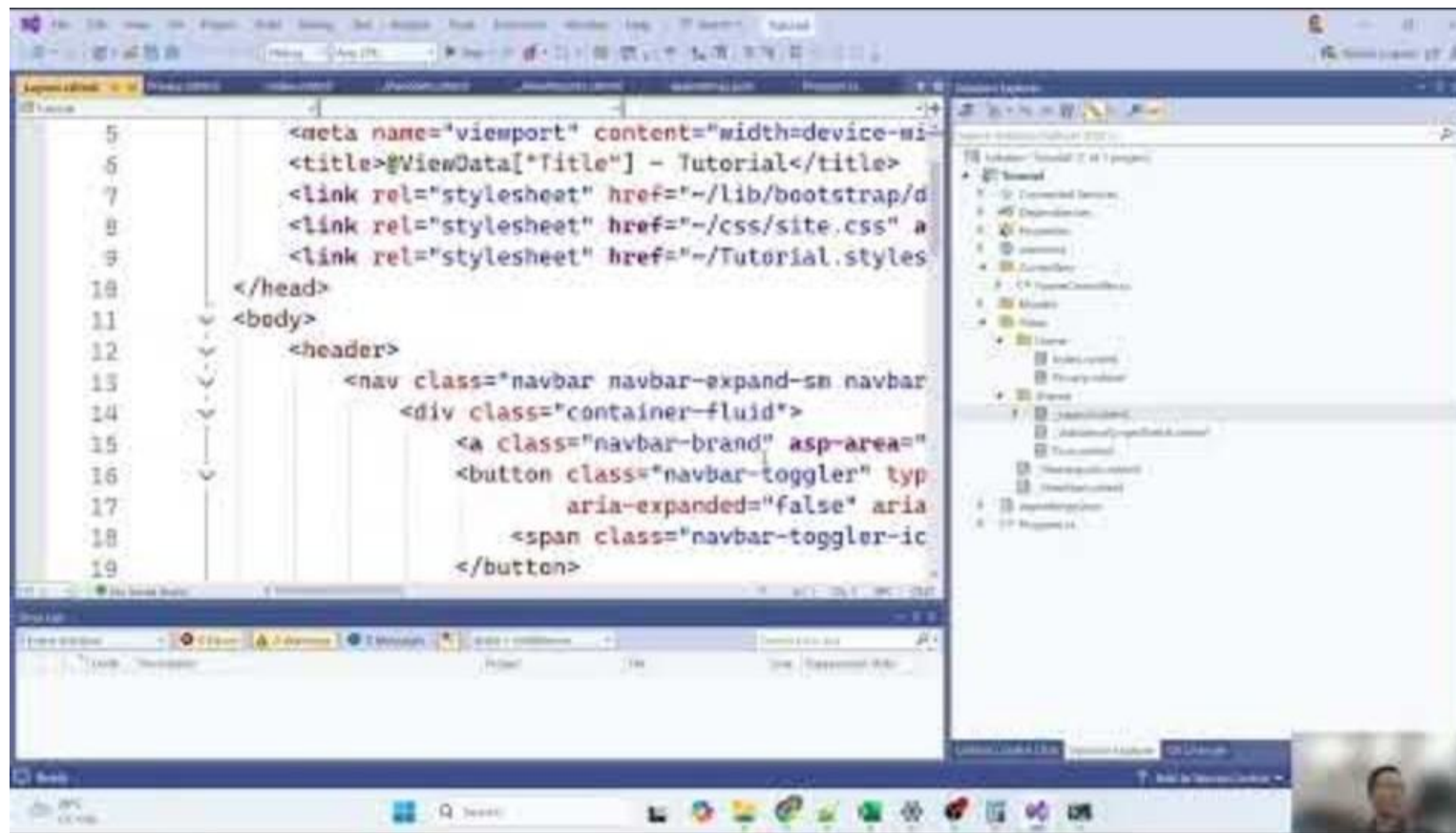
Mối quan hệ 1-n, n-n



Dễ mapping domain



Kiểm thử logic ngay



<https://youtu.be/Fxhl1n1H6sU>



Tổng kết



CDR = checkpoint kỹ thuật cuối



Kiểm Use–Seq–Class



Áp dụng 10 nguyên tắc



Kỷ luật thiết kế nghiêm ngặt



Chuyển mô hình → code EF Core



Bài tập tổng kết



Chọn Use
Case bất kỳ



Vẽ
Sequence &
Class



Tổ chức
mini-CDR
nhóm 4 người



Ghi
checklist 10
nguyên tắc



Sinh code
bằng EF Core